

```
In [ ]: import marimo as mo
```

```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nibabel as nib
import pandas as pd

from numpy.fft import fft, ifft, fftfreq
from scipy.signal import butter, filtfilt
# from nltools.stats import regress
# from nltools.data import Design_Matrix
from dartbrains_tools.notebook_utils import plot_timeseries

from sklearn.decomposition import FastICA

import statsmodels.api as sm
```

```
In [ ]: # function to find spikes copied from nltools
```

```
def find_spikes(data, global_spike_cutoff=3, diff_spike_cutoff=3):
    """
    Function to identify spikes from a 1D timeseries.

    Parameters:
        data: 1D timeseries data
        global_spike_cutoff: cutoff to identify spikes in global signal in s
        diff_spike_cutoff: cutoff to identify spikes in average frame differ
        global_spike_cutoff: cutoff to identify spikes in global signal in s
        diff_spike_cutoff: cutoff to identify spikes in average frame differ

    Returns:
        spikes_df: pandas dataframe with spikes as indicator variables
    """

    data = np.array(data).squeeze()
    frame_diff = np.abs(np.diff(data))

    if global_spike_cutoff is not None:
        global_outliers = np.append(
            np.where(data > np.mean(data) + np.std(data) * global_spike_cuto
            np.where(data < np.mean(data) - np.std(data) * global_spike_cuto
            )

    if diff_spike_cutoff is not None:
        frame_outliers = np.append(
            np.where(frame_diff > np.mean(frame_diff) + np.std(frame_diff) *
            np.where(frame_diff < np.mean(frame_diff) - np.std(frame_diff) *

    # build spike regressors
    spikes_df = pd.DataFrame(index=range(len(data)))
    if global_spike_cutoff is not None:
        for i, loc in enumerate(global_outliers):
```

```

        col = f"global_spike{str(i + 1)}"
        spikes_df[col] = 0
        spikes_df.loc[int(loc), col] = 1
    # build FD regressors
    if diff_spike_cutoff is not None:
        for i, loc in enumerate(frame_outliers):
            col = f"diff_spike{str(i + 1)}"
            spikes_df[col] = 0
            spikes_df.loc[int(loc), col] = 1
    return spikes_df

```

```

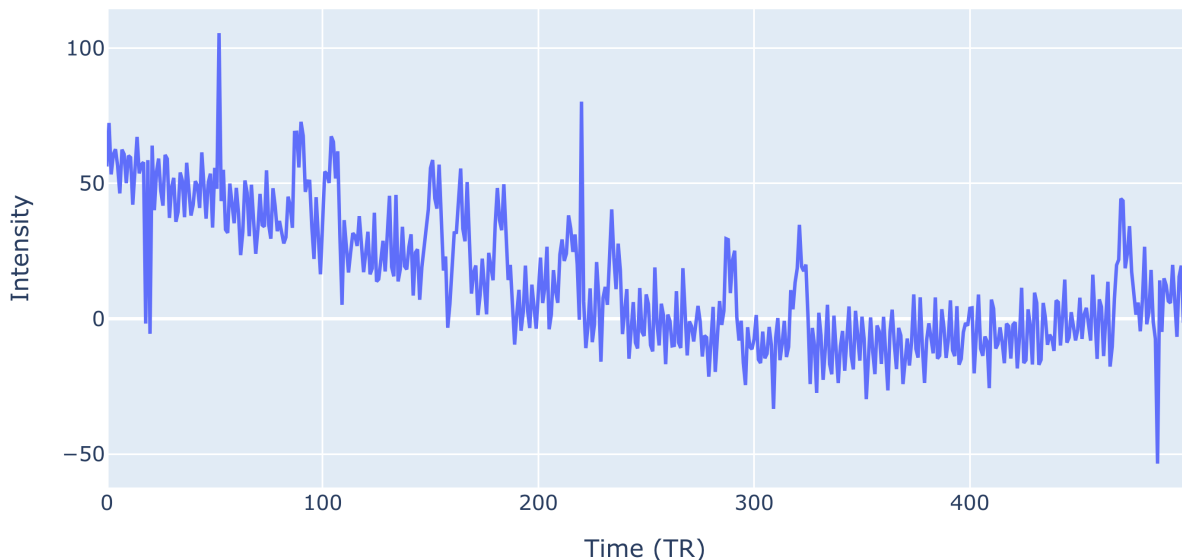
In [ ]: TR = 2.0
        hp_cutoff = 0.01
        poly_order = 2

        data = pd.read_csv('Noisy_Signal.csv')
        original = data.copy()
        signal = data.values.squeeze()
        n = len(signal)
        print(f"Original signal has {n} time points.")

        plot_timeseries(data)

```

Original signal has 500 time points.



```

In [ ]: # intercept
        intercept = np.ones((n,1))
        intercept.shape

```

```

In [ ]: # find global and local spikes
        spikes = find_spikes(data).values
        spikes.shape

```

```

In [ ]: # DCT high-pass
        n_dct      = int(np.floor(2 * n * TR * hp_cutoff))
        timepoints = np.arange(n)
        k           = np.arange(1, n_dct + 1)

```

```
dct      = np.sqrt(2 / n) * np.cos(
    np.pi * k[np.newaxis, :] * (2 * timepoints[:, np.newaxis] + 1) / (2 * n)
)
dct.shape
```

```
In [ ]: # polynomial trend regressors
t      = np.linspace(-1, 1, n)
poly = np.column_stack([t ** i for i in range(1, poly_order + 1)]) # shape
poly.shape
```

```
In [ ]: # Combine everything
X = np.hstack([intercept, spikes, dct, poly]) # shape (n, 1 + n_spikes + n_
X.shape
```

```
In [ ]: model = sm.OLS(signal, X).fit()

print(model.summary())
betas = model.params
```

OLS Regression Results

```

=====
==
Dep. Variable:          y    R-squared:          0.7
65
Model:                  OLS   Adj. R-squared:       0.7
49
Method:                 Least Squares   F-statistic:       47.
55
Date:                   Mon, 11 May 2026   Prob (F-statistic): 9.32e-1
26
Time:                   01:35:11   Log-Likelihood:    -194
6.6
No. Observations:      500   AIC:               395
9.
Df Residuals:          467   BIC:               409
8.
Df Model:               32
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975
5]						

const	7.1046	39.411	0.180	0.857	-70.340	84.5
49						
x1	32.0495	6.290	5.096	0.000	19.690	44.4
09						
x2	4.5034	12.731	0.354	0.724	-20.514	29.5
21						
x3	-54.8491	12.711	-4.315	0.000	-79.827	-29.8
71						
x4	5.7534	12.690	0.453	0.650	-19.183	30.6
90						
x5	-58.1749	12.669	-4.592	0.000	-83.071	-33.2
79						
x6	6.1832	12.575	0.492	0.623	-18.527	30.8
93						
x7	32.0495	6.290	5.096	0.000	19.690	44.4
09						
x8	-16.6691	12.559	-1.327	0.185	-41.349	8.0
11						
x9	64.0546	12.561	5.100	0.000	39.372	88.7
38						
x10	-18.8551	12.742	-1.480	0.140	-43.893	6.1
83						
x11	-64.5931	12.742	-5.069	0.000	-89.632	-39.5
55						
x12	-2045.2310	2817.827	-0.726	0.468	-7582.421	3491.9
59						
x13	57.5687	757.919	0.076	0.939	-1431.784	1546.9
22						
x14	-317.5458	313.814	-1.012	0.312	-934.207	299.1
16						
x15	13.7731	190.227	0.072	0.942	-360.034	387.5

81						
x16	-110.6836	113.888	-0.972	0.332	-334.479	113.1
12						
x17	23.2465	85.501	0.272	0.786	-144.768	191.2
61						
x18	-56.2470	59.229	-0.950	0.343	-172.635	60.1
41						
x19	0.7491	49.268	0.015	0.988	-96.065	97.5
64						
x20	-20.9769	37.274	-0.563	0.574	-94.222	52.2
68						
x21	23.7286	32.953	0.720	0.472	-41.026	88.4
83						
x22	-12.0337	26.563	-0.453	0.651	-64.232	40.1
64						
x23	19.7299	24.533	0.804	0.422	-28.478	67.9
38						
x24	25.9884	20.828	1.248	0.213	-14.940	66.9
17						
x25	-14.0159	19.796	-0.708	0.479	-52.916	24.8
85						
x26	-32.3897	17.536	-1.847	0.065	-66.848	2.0
69						
x27	-48.7310	17.089	-2.852	0.005	-82.312	-15.1
50						
x28	-15.1305	15.638	-0.968	0.334	-45.860	15.5
99						
x29	15.5792	15.435	1.009	0.313	-14.752	45.9
11						
x30	20.3588	14.520	1.402	0.162	-8.173	48.8
91						
x31	-7.7466	14.433	-0.537	0.592	-36.108	20.6
14						
x32	-190.5822	219.377	-0.869	0.385	-621.670	240.5
06						
x33	21.5122	117.693	0.183	0.855	-209.761	252.7
86						

=====						
==						
Omnibus:		9.523	Durbin-Watson:			1.3
76						
Prob(Omnibus):		0.009	Jarque-Bera (JB):			9.6
37						
Skew:		0.300	Prob(JB):			0.008
08						
Kurtosis:		3.322	Cond. No.			2.25e+
15						
=====						
==						

Notes:

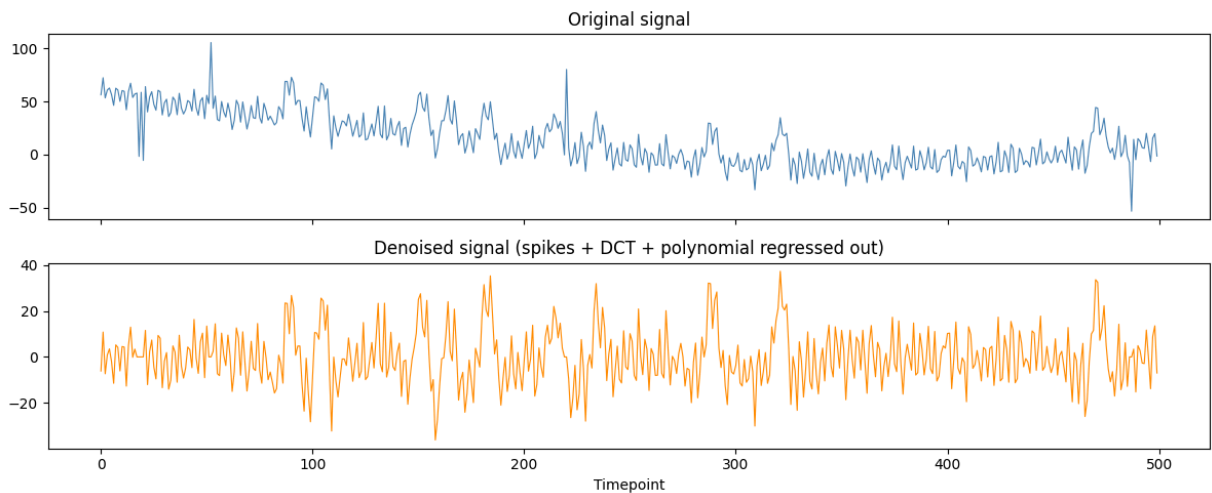
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.11e-28. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [ ]: signal_clean = signal - X @ betas # residuals (nuisance removed, mean als
```

```
In [ ]: # write out signal_clean to csv  
pd.DataFrame(signal_clean, columns=['Denoised_Signal']).to_csv('Denoised_Sig
```

```
In [ ]: fig, axes = plt.subplots(2, 1, figsize=(12, 5), sharex=True)  
axes[0].plot(original.values, color='steelblue', lw=0.8)  
axes[0].set_title('Original signal')  
axes[1].plot(signal_clean, color='darkorange', lw=0.8)  
axes[1].set_title('Denoised signal (spikes + DCT + polynomial regressed out)')  
axes[1].set_xlabel('Timepoint')  
fig.tight_layout()  
plt.savefig('denoised_signal.png', dpi=150)  
plt.show()
```



Writeup

I used global and local spikes, discrete cosine transforms, and polynomial drift nuisance variables to regress out the noise in the signal. The denoised signal can be found in

`Denoised_Signal.csv`. Plot above!